ΜΑΘΗΜΑ 2 ΑΠΟ 15 · ΥΠΟΕΝΟΤΗΤΑ 2.3.Γ.1

if · while · for

Το πρόγραμμα αποφασίζει και επαναλαμβάνει

Θεωρία + 3 ασκήσεις + 6 προγράμματα με λύσεις Διάρκεια: 4 ώρες (3η–6η) 26 διαφάνειες

Προγραμματισμός Εφαρμογών Διαδικτύου & Παιχνιδιών (Python, Pygame) · Γ΄ εξάμηνο · Δευτέρα 12/10/2026

Τι θα ξέρεις στο τέλος

1. Να γράφεις αποφάσεις με **if / elif / else** — και να ξέρεις ότι εκτελείται **μόνο ένας** κλάδος.
2. Να συνδυάζεις συνθήκες με **and / or / not** και να γράφεις **ένθετα if**.
3. Να γράφεις βρόχο **while** με τα τρία μέρη του: αρχικοποίηση, συνθήκη, μεταβολή.
4. Να χρησιμοποιείς **for** με **range()** και πάνω σε συμβολοσειρά.
5. Να ελέγχεις τη ροή με **break**, **continue** και **else** σε βρόχο.
6. Να γράφεις ένα πλήρες πρόγραμμα με **μενού** που τρέχει μέχρι ο χρήστης να πατήσει 0.

Πώς είναι οργανωμένο το 4ωρο

Ώρες 1–2: επιλογή (if) και επανάληψη (while, for) με live coding. **Ώρες 3–4:** εργαστήριο — προπαίδεια, πρώτοι αριθμοί, «Μάντεψε τον αριθμό» και στο τέλος ένα **μενού κονσόλας**.

ΣΥΝΈΧΕΙΑ Τα προγράμματα της προηγούμενης φοράς (ΔΜΣ, ρέστα) σήμερα αποκτούν «μυαλό»: θα παίρνουν αποφάσεις.

Θυμάσαι από την προηγούμενη φορά;

ΕΚΦΏΝΗΣΗ

Πέντε γρήγορες ερωτήσεις, 5 λεπτά. Πρώτα απάντηση στο χαρτί, μετά έλεγχος στη γραμμή εντολών της Python.

ΛΎΣΗ

ΈΚΦΡΑΣΗ	ΑΠΟΤΈΛΕΣΜΑ	ΓΙΑΤΊ
`7 // 2`	`3`	ακέραιο πηλίκο
`17 % 5`	`2`	υπόλοιπο
`"5" + "3"`	`"53"`	κείμενα: κολλάνε
`type(10 / 2)`	`float`	η / δίνει πάντα float
`int("12") + 1`	`13`	πρώτα μετατροπή, μετά πράξη

Γιατί τα ξαναθυμόμαστε

Σήμερα θα γράφουμε **συνθήκες** όπως **n % 2 == 0** και θα διαβάζουμε επιλογές με **input()**. Αν αυτά δεν είναι σίγουρα, το if δεν θα δουλέψει.

ΘΥΜΉΣΟΥ Η input() επιστρέφει **πάντα κείμενο**. Μια επιλογή μενού "1" είναι str — θα το χρειαστούμε στο τέλος.

if: κάνε κάτι μόνο αν ισχύει

```
if συνθήκη:
    εντολή 1
    εντολή 2
εντολή μετά το if
```

```
age = int(input("Ηλικία: "))

if age >= 18:
    print("Ενήλικας")
    print("Μπορείς να ψηφίσεις")

print("Τέλος")
```

Πώς δουλεύει

Αν η συνθήκη είναι **True**, εκτελούνται οι γραμμές με εσοχή. Αν είναι **False**, παραλείπονται.
Η γραμμή χωρίς εσοχή εκτελείται **πάντα**.

ΕΊΣΟΔΟΣ	AGE >= 18	ΕΜΦΑΝΊΖΕΙ
25	True	Ενήλικας · Μπορείς να ψηφίσεις · Τέλος
15	False	μόνο: Τέλος

ΤΑ ΔΎΟ ΠΙΟ ΣΥΧΝΆ ΛΆΘΗ Ξεχνιέται η **άνω-κάτω τελεία** (:) στο τέλος του if, ή μπαίνει = αντί για == στη σύγκριση.

if … else: ή το ένα ή το άλλο

```
n = int(input("Αριθμός: "))

if n % 2 == 0:
    print("Άρτιος")
else:
    print("Περιττός")
```

N	N % 2	ΑΠΟΤΈΛΕΣΜΑ
8	0	Άρτιος
13	1	Περιττός
0	0	Άρτιος

Το else

Δεν έχει δική του συνθήκη. Εκτελείται όταν η συνθήκη του if είναι **False**. Έτσι εκτελείται **πάντα ακριβώς ένα** από τα δύο μπλοκ — ποτέ και τα δύο, ποτέ κανένα.

Θυμάσαι το %;

Το **n % 2 == 0** είναι ο κλασικός έλεγχος «διαιρείται ακριβώς». Με **n % 5 == 0** ελέγχεις αν διαιρείται με το 5.

ΣΤΟΊΧΙΣΗ Το **else** μπαίνει στην **ίδια εσοχή** με το if στο οποίο ανήκει — όχι μέσα στο μπλοκ του.

if … elif … else: πολλές περιπτώσεις

```
grade = float(input("Βαθμός (0-20): "))

if grade >= 18.5:
    print("Άριστα")
elif grade >= 16:
    print("Πολύ καλά")
elif grade >= 13:
    print("Καλά")
elif grade >= 10:
    print("Μέτρια")
else:
    print("Κόπηκε")
```

Η Python ελέγχει από πάνω προς τα κάτω

Μόλις βρει την **πρώτη** συνθήκη που ισχύει, εκτελεί εκείνο το μπλοκ και **πετάει όλα τα υπόλοιπα**.

Γι' αυτό η σειρά έχει σημασία

Αν βάζαμε πρώτο το **grade >= 10**, ένα 19 θα έβγαζε «Μέτρια»: ισχύει κι αυτό, και θα σταματούσε εκεί.

ΒΑΘΜΌΣ	ΠΡΏΤΗ ΣΥΝΘΉΚΗ ΠΟΥ ΙΣΧΎΕΙ
17	grade >= 16 → Πολύ καλά
9.5	καμία → else → Κόπηκε

ΓΙΑΤΊ ΌΧΙ `13 <= grade < 16` Δεν χρειάζεται: αν φτάσαμε στο elif grade >= 13, ξέρουμε ήδη ότι ο βαθμός είναι **κάτω από 16**.

Ένθετα if ή and;

ΈΝΘΕΤΑ IF

```
user = input("Χρήστης: ")
pwd = input("Κωδικός: ")
if user == "admin":
    if pwd == "1234":
        print("Καλώς ήρθες")
    else:
        print("Λάθος κωδικός")
else:
    print("Άγνωστος χρήστης")
```

ΜΕ AND

```
user = input("Χρήστης: ")
pwd = input("Κωδικός: ")
if user == "admin" and pwd == "1234":
    print("Καλώς ήρθες")
else:
    print("Λάθος στοιχεία")
```

Πότε ένθετα

Όταν κάθε αποτυχία θέλει **δικό της μήνυμα**: άλλο για άγνωστο χρήστη, άλλο για λάθος κωδικό.

Πότε and

Όταν σε ενδιαφέρει μόνο αν **ισχύουν όλα μαζί**. Ο κώδικας είναι πιο σύντομος και πιο καθαρός.

Θυμήσου τους λογικούς τελεστές

and = και τα δύο · **or** = έστω ένα · **not** = το αντίθετο. Π.χ. **if not (1 <= day <= 7):** → «λάθος ημέρα».

ΚΑΛΉ ΠΡΑΚΤΙΚΉ Πάνω από 2–3 επίπεδα εσοχής δύσκολα διαβάζονται. Αν φτάσεις εκεί, δες μήπως λύνεται με and ή elif.

if σε μία γραμμή

```python
# η κανονική μορφή
if grade >= 10:
    result = "Πέρασε"
else:
    result = "Κόπηκε"
```

```python
# σε μία γραμμή
result = "Πέρασε" if grade >= 10 else "Κόπηκε"
```

Η μορφή

τιμή_αν_True if συνθήκη else τιμή_αν_False — διαβάζεται σαν πρόταση: «Πέρασε, αν ο βαθμός είναι ≥ 10, αλλιώς Κόπηκε».

ΠΑΡΑΔΕΊΓΜΑΤΑ

ΚΩΔΙΚΑΣ	ΤΙ ΚΆΝΕΙ
`bigger = a if a > b else b`	ο μεγαλύτερος από δύο (υπάρχει και έτοιμο: max(a, b))
`kind = "άρτιος" if n % 2 == 0 else "περιττός"`	λέξη ανάλογα με τον αριθμό
`print("Ναι" if ok else "Όχι")`	και μέσα σε print

ΠΌΤΕ ΝΑ ΤΟ ΑΠΟΦΎΓΕΙΣ Μόνο για **απλές** επιλογές ανάμεσα σε δύο τιμές. Για δύο ή περισσότερες εντολές, γράψε κανονικό if.

Τι θα τυπώσει;

ΕΚΦΏΝΗΣΗ

Τρέξε τα δύο κομμάτια με το μυαλό για τις τιμές του πίνακα. Η μόνη διαφορά τους: **elif** ή δεύτερο **if**.

ΚΟΜΜΆΤΙ 1 · ELIF

```
if x > 5:
    print("A")
elif x > 3:
    print("B")
else:
    print("Γ")
```

ΚΟΜΜΆΤΙ 2 · ΔΎΟ IF

```
if x > 5:
    print("A")
if x > 3:
    print("B")
```

ΛΎΣΗ

X	ΚΟΜΜΆΤΙ 1	ΚΟΜΜΆΤΙ 2
7	A	**A και B**
4	B	B
2	Γ	(τίποτα)

Η διαφορά

Με **elif** οι περιπτώσεις **αποκλείουν** η μία την άλλη. Με δύο **if** ελέγχονται **ανεξάρτητα** — για x = 7 ισχύουν και οι δύο.

ΚΑΝΌΝΑΣ Αν οι περιπτώσεις δεν πρέπει να συμβούν μαζί (βαθμοί, κατηγορίες), θέλεις **elif**.

Ο ΔΜΣ αποκτά κατηγορία

ΕΚΦΏΝΗΣΗ

Συμπλήρωσε το πρόγραμμα ΔΜΣ της προηγούμενης φοράς ώστε να εμφανίζει και την κατηγορία.

Όρια ΠΟΥ: < 18.5 · < 25 · < 30 · από 30 και πάνω.

ΔΜΣ	ΚΑΤΗΓΟΡΊΑ
κάτω από 18.5	λιποβαρής
18.5 – 24.9	φυσιολογικό βάρος
25 – 29.9	υπέρβαρος
30 και πάνω	παχυσαρκία

ΛΎΣΗ

```
weight = float(input("Βάρος σε kg: "))
height = float(input("Ύψος σε m: "))
bmi = weight / height ** 2

if bmi < 18.5:
    category = "λιποβαρής"
elif bmi < 25:
    category = "φυσιολογικό βάρος"
elif bmi < 30:
    category = "υπέρβαρος"
else:
    category = "παχυσαρκία"
print(f"ΔΜΣ: {bmi:.2f} ({category})")
```

```
Βάρος σε kg: 70
Ύψος σε m: 1.75
ΔΜΣ: 22.86 (φυσιολογικό βάρος)
```

Γιατί κρατάμε την κατηγορία σε μεταβλητή

Μία print στο τέλος αντί για τέσσερις. Αν αλλάξει η μορφή της εξόδου, την αλλάζεις σε **ένα** σημείο.

ΠΑΡΑΤΉΡΗΣΗ Ανεβαίνουμε από το μικρότερο όριο (< 18.5) προς το μεγαλύτερο. Με < κάθε elif κρύβει από πριν το «≥ προηγούμενο όριο».

Τα ρέστα ελέγχουν την πληρωμή

ΕΚΦΏΝΗΣΗ

Τι γίνεται στο πρόγραμμα των ρέστων αν ο πελάτης δώσει **λιγότερα** χρήματα; Πρόσθεσε έλεγχο **πριν** από τον υπολογισμό των κερμάτων.

ΛΎΣΗ

```
price = float(input("Ποσό (€): "))
paid = float(input("Πληρωμή (€): "))

if paid < price:
    print(f"Λείπουν {price - paid:.2f}€")
elif paid == price:
    print("Ακριβές ποσό, χωρίς ρέστα")
else:
    rest = round((paid - price) * 100)
    print(f"Ρέστα: {rest / 100:.2f}€")
    # ... τα κέρματα όπως την προηγούμενη φορά
```

ΠΟΣΌ	ΠΛΗΡΩΜΉ	ΑΠΟΤΈΛΕΣΜΑ
15.12	10	Λείπουν 5.12€
15.00	15	Ακριβές ποσό
15.12	20	Ρέστα: 4.88€ + κέρματα

Όλος ο παλιός κώδικας μπαίνει στο else

Με **μία εσοχή** πιο μέσα. Στο VS Code: μαρκάρεις τις γραμμές και πατάς **Tab**.

ΜΆΘΗΜΑ Ένα πρόγραμμα χωρίς έλεγχο δουλεύει μόνο όταν ο χρήστης κάνει τα σωστά. Το if το κάνει να αντέχει και τα λάθη.

Γιατί χρειαζόμαστε βρόχο

Θέλουμε να τυπώσουμε τους αριθμούς από το 1 έως το 5. Μετά από το 1 έως το 1000.

ΧΩΡΊΣ ΕΠΑΝΆΛΗΨΗ

```
print(1)
print(2)
print(3)
print(4)
print(5)
# ... για 1000 αριθμούς;
```

ΜΕ ΕΠΑΝΆΛΗΨΗ

```
for i in range(1, 6):
    print(i)

# για 1000 αλλάζει ένας αριθμός:
for i in range(1, 1001):
    print(i)
```

while

Επανάλαβε **όσο** ισχύει μια συνθήκη. Για όταν **δεν ξέρεις** πόσες φορές.

for

Επανάλαβε **για κάθε** στοιχείο μιας σειράς. Για όταν **ξέρεις** πόσες φορές ή τι διατρέχεις.

Δεν υπάρχει

Η Python **δεν έχει do-while**. Το «τουλάχιστον μία φορά» γίνεται με while True + break.

Η ΟΥΣΊΑ Ο βρόχος δεν είναι κόλπο για λιγότερο γράψιμο. Είναι ο μόνος τρόπος να δουλεύει ο κώδικας για πλήθος που δεν ξέρεις από πριν.

while: όσο ισχύει, ξανά

```
while συνθήκη:
    εντολές
    # κάτι πρέπει να αλλάζει τη συνθήκη!
```

```
i = 1                # 1. αρχικοποίηση
while i <= 3:        # 2. συνθήκη
    print(i)
    i += 1           # 3. μεταβολή
print("Τέλος")
```

Πώς δουλεύει

Ελέγχει **πρώτα** τη συνθήκη. Όσο είναι **True**, εκτελεί το μπλοκ και ξαναελέγχει. Μόλις γίνει **False**, βγαίνει.

ΓΎΡΟΣ	I	I <= 3	ΤΥΠΏΝΕΙ
1	1	True	**1**
2	2	True	**2**
3	3	True	**3**
—	4	False	Τέλος (έξοδος)

1. Αρχικοποίηση

Πριν τον βρόχο. Δίνει την πρώτη τιμή στη μεταβλητή που ελέγχεται.

2. Συνθήκη

Στην κορυφή. Πρέπει **κάποτε** να γίνει False.

3. Μεταβολή

Μέσα στον βρόχο. Αν λείπει → **ατέρμονας βρόχος**.

ΚΡΊΣΙΜΟ Αν η συνθήκη είναι False από την αρχή, ο βρόχος εκτελείται **μηδέν** φορές.

while με άγνωστο πλήθος

Ο χρήστης δίνει αριθμούς και σταματά με το 0. Δεν ξέρουμε από πριν πόσους θα δώσει.

```
total = 0
n = int(input("Αριθμός (0 = τέλος): "))
while n != 0:
    total += n
    n = int(input("Αριθμός (0 = τέλος): "))
print("Άθροισμα:", total)
```

```
Αριθμός (0 = τέλος): 5
Αριθμός (0 = τέλος): 12
Αριθμός (0 = τέλος): 3
Αριθμός (0 = τέλος): 0
Άθροισμα: 20
```

Γιατί η input() γράφεται δύο φορές

Η πρώτη δίνει τιμή **πριν** ελεγχθεί η συνθήκη. Η δεύτερη, στο τέλος του μπλοκ, είναι η **μεταβολή**: χωρίς αυτήν το n δεν αλλάζει ποτέ.

Ατέρμονας βρόχος;

Αν το πρόγραμμα «κόλλησε» και τυπώνει ασταμάτητα, πάτα **Ctrl+C** στο Terminal. Μετά ψάξε ποια μεταβλητή δεν αλλάζει.

Το 0 είναι «φρουρός» (sentinel)

Μια ειδική τιμή που δεν είναι δεδομένο αλλά σήμα «τέλος». Γι' αυτό **δεν προστίθεται** στο άθροισμα.

ΓΙΑΤΊ WHILE Εδώ το for δεν ταιριάζει: δεν ξέρουμε πόσους αριθμούς θα δώσει ο χρήστης.

for με range()

```
for i in range(1, 6):
    print(i, end=" ")
1 2 3 4 5
```

Τι κάνει μόνο του

Δίνει στο **i** κάθε τιμή του range με τη σειρά. Αρχικοποίηση, έλεγχος και μεταβολή γίνονται **αυτόματα**.

ΚΛΉΣΗ	ΤΙΜΈΣ ΤΟΥ I	ΚΑΝΌΝΑΣ
`range(5)`	`0 1 2 3 4`	από 0, **5 φορές**
`range(1, 6)`	`1 2 3 4 5`	από 1 έως 6, **χωρίς το 6**
`range(0, 11, 2)`	`0 2 4 6 8 10`	βήμα 2
`range(10, 0, -1)`	`10 9 8 7 6 5 4 3 2 1`	αρνητικό βήμα: αντίστροφα
`range(5, 1)`	(καμία)	η αρχή ήδη μετά το τέλος

Ο χρυσός κανόνας του range

Το **τέλος δεν περιλαμβάνεται** ποτέ. Για 1 έως 10 γράφεις **range(1, 11)**. Πλήθος επαναλήψεων με βήμα 1: **τέλος − αρχή**.

ΣΥΧΝΌ ΛΆΘΟΣ Να γράψεις range(1, 10) και να περιμένεις και το 10. Θα σταματήσει στο 9.

for πάνω σε συμβολοσειρά

```
for ch in "Γεια":
    print(ch)
Γ
ε
ι
α
```

Το for της Python

Το for δεν μετράει απαραίτητα αριθμούς. **Διατρέχει** ό,τι του δώσεις: ένα range, ένα κείμενο, και σε δύο εβδομάδες λίστες.
Το **ch** παίρνει ένα-ένα τα γράμματα.

```
word = "Python"
count = 0
for ch in word:
    if ch in "aeiouy":
        count += 1
print(f"Φωνήεντα: {count}")
Φωνήεντα: 2
```

```
for i, ch in enumerate("Py", start=1):
    print(i, ch)
1 P
2 y
```

if μέσα σε for

Συνδυάζουμε τις δύο δομές: για κάθε γράμμα, **αν** είναι φωνήεν, μέτρα το.

enumerate()

Όταν θέλεις **και θέση και στοιχείο** μαζί. Το start=1 ξεκινά την αρίθμηση από το 1.

ΧΡΉΣΙΜΟ Το **len(word)** δίνει το μήκος. Το **ch in "aeiou"** ελέγχει αν ένα γράμμα ανήκει σε μια ομάδα γραμμάτων.

Αθροιστής και μετρητής

```
total = 0          # αθροιστής
evens = 0          # μετρητής

for i in range(1, 101):
    total += i
    if i % 2 == 0:
        evens += 1
print(total, evens)
5050 50
```

Εδώ χάνονται οι περισσότεροι

Η αρχικοποίηση μπαίνει **έξω** από τον βρόχο. Αν μπει μέσα, μηδενίζεται σε κάθε γύρο και το αποτέλεσμα βγαίνει λάθος.

Αθροιστής

Ξεκινά από **0** και μαζεύει τιμές: **total += i**.

Μετρητής

Ξεκινά από **0** και αυξάνεται κατά **1** μόνο όταν ισχύει κάτι: **evens += 1**.

Έτοιμη συνάρτηση

Για απλό άθροισμα υπάρχει και το **sum(range(1, 101))**. Τον βρόχο όμως τον χρειάζεσαι όταν μπαίνουν συνθήκες.

ΙΣΤΟΡΊΑ Ο Gauss, μαθητής, βρήκε το 5050 χωρίς βρόχο: 100 · 101 / 2. Ο υπολογιστής απλώς το κάνει με «ωμή βία».

Τι θα τυπώσει ο βρόχος;

ΕΚΦΏΝΗΣΗ

Γράψε τι εμφανίζει κάθε βρόχος (στη σειρά, με κενά). Για το while φτιάξε **πίνακα τιμών**.

ΛΎΣΗ

ΚΩΔΙΚΑΣ	ΤΥΠΏΝΕΙ	ΣΧΌΛΙΟ
`for i in range(1, 10, 3): print(i)`	`1 4 7`	βήμα 3
`for i in range(3): print(i)`	`0 1 2`	από το 0
`for i in range(5, 0): print(i)`	(τίποτα)	αρχή > τέλος
`for i in range(10, 0, -3): print(i)`	`10 7 4 1`	αντίστροφα
`for ch in "abc": print(ch * 2)`	`aa bb cc`	επανάληψη κειμένου

```
x = 10
while x > 0:
    x -= 3
    print(x)
7 4 1 -2
```

Η παγίδα του while

Τυπώνει **-2**! Το print είναι **μετά** τη μεταβολή: όταν x = 1, ο έλεγχος περνάει, το x γίνεται -2 και τυπώνεται.

ΤΕΧΝΙΚΉ Στο while γράφε πίνακα με στήλες: x πριν · συνθήκη · x μετά · τι τυπώνει. Δύο λεπτά δουλειάς σε σώζουν από λάθη.

break και continue

BREAK · ΣΤΑΜΆΤΑ ΤΟΝ ΒΡΌΧΟ

```
for n in range(50, 100):
    if n % 7 == 0:
        print("Βρέθηκε:", n)
        break

Βρέθηκε: 56
```

break

Μόλις βρεθεί το πρώτο πολλαπλάσιο του 7, **δεν έχει νόημα** να συνεχίσουμε. Ο βρόχος τελειώνει αμέσως.

CONTINUE · ΠΉΓΑΙΝΕ ΣΤΟΝ ΕΠΌΜΕΝΟ ΓΎΡΟ

```
for n in range(1, 11):
    if n % 3 == 0:
        continue
    print(n, end=" ")

1 2 4 5 7 8 10
```

continue

Τα πολλαπλάσια του 3 **παραλείπονται**: ό,τι ακολουθεί στο μπλοκ δεν εκτελείται για αυτόν τον γύρο.

```
while True:
    pin = input("PIN: ")
    if pin == "2468":
        break
    print("Λάθος, ξανά")
```

while True + break

Ο βρόχος που τρέχει **τουλάχιστον μία φορά**: ρώτα, και βγες μόλις η απάντηση είναι σωστή.

ΣΗΜΕΊΩΣΗ Και το break και το continue επηρεάζουν **μόνο τον πιο εσωτερικό** βρόχο στον οποίο βρίσκονται.

else σε βρόχο: «δεν βρέθηκε»

```
word = input("Λέξη: ")

for ch in word:
    if ch == "y":
        print("Έχει y")
        break
else:
    print("Δεν έχει y")
```

Η στοίχιση λέει τα πάντα

Το else είναι στην **ίδια εσοχή με το for**, όχι με το if. Αν το μετακινήσεις κάτω από το if, αλλάζει εντελώς νόημα.

Πότε εκτελείται το else

Μόνο όταν ο βρόχος τελειώσει **κανονικά**, χωρίς break. Δηλαδή όταν ψάξαμε παντού και **δεν βρήκαμε**.

ΛΈΞΗ	BREAK;	ΕΜΦΑΝΙΖΕΙ
python	ναι	Έχει y
java	όχι	Δεν έχει y

Θα το χρειαστούμε αμέσως

Στον έλεγχο **πρώτου αριθμού**: «δεν βρέθηκε διαιρέτης» = πρώτος.

ΣΥΝΤΟΜΟΓΡΑΦΊΑ Για αυτό το συγκεκριμένο παράδειγμα υπάρχει και το **"y" in word**. Το for-else χρειάζεται όταν ο έλεγχος είναι πιο σύνθετος.

for ή while;

ΕΚΦΏΝΗΣΗ

Ποια δομή επανάληψης ταιριάζει; α) Τύπωσε την προπαίδεια του 7. β) Ζήτα PIN μέχρι να δοθεί το σωστό. γ) Άθροισε τους αριθμούς 1 έως 100. δ) Διάβαζε τιμές μέχρι ο χρήστης να δώσει 0. ε) Μέτρα τα κεφαλαία γράμματα μιας πρότασης.

ΛΎΣΗ

α · for	Γνωστό πλήθος: **10** γινόμενα → for i in range(1, 11).
β · while	Δεν ξέρουμε **πόσες** προσπάθειες θα χρειαστούν → while True + break.
γ · for	Γνωστό εύρος → for i in range(1, 101).
δ · while	Άγνωστο πλήθος, σταματά με **φρουρό** → while n != 0.
ε · for	Διατρέχουμε **κάθε γράμμα** του κειμένου → for ch in text.

Ο ΚΑΝΌΝΑΣ Ξέρω **από πριν** πόσες φορές ή τι διατρέχω; → **for**. Εξαρτάται από κάτι που θα συμβεί στην πορεία; → **while**.

Προπαίδεια

ΕΚΦΏΝΗΣΗ

Διάβασε έναν αριθμό n και εμφάνισε την προπαίδειά του από το 1 έως το 10, **στοιχισμένη**.

ΛΎΣΗ

```
n = int(input("Αριθμός: "))

for i in range(1, 11):
    print(f"{n} x {i:2d} = {n * i:3d}")
```

ΠΡΌΚΛΗΣΗ · ΌΛΗ Η ΠΡΟΠΑΊΔΕΙΑ ΜΕ ΈΝΘΕΤΟΥΣ ΒΡΌΧΟΥΣ

```
for row in range(1, 11):
    for col in range(1, 11):
        print(f"{row * col:4d}", end="")
    print()
```

```
Αριθμός: 7
7 x  1 =   7
7 x  2 =  14
...
7 x 10 =  70
```

Το :2d και το :3d

Κρατάνε **σταθερό πλάτος** ψηφίων, ώστε τα = να μπαίνουν το ένα κάτω από το άλλο.

Βρόχος μέσα σε βρόχο

Για **κάθε** γραμμή, ο εσωτερικός βρόχος τρέχει **10 φορές**. Σύνολο 10 × 10 = 100 εκτυπώσεις.
Το άδειο **print()** αλλάζει γραμμή.

ΓΙΑ ΤΟΥΣ ΓΡΉΓΟΡΟΥΣ Βάλε και επικεφαλίδα: μια πρώτη γραμμή με τους αριθμούς 1 έως 10 πάνω από τον πίνακα.

Είναι πρώτος;

ΕΚΦΏΝΗΣΗ

Διάβασε έναν ακέραιο και πες αν είναι **πρώτος** (διαιρείται μόνο με το 1 και τον εαυτό του). Αν δεν είναι, δείξε έναν διαιρέτη του.

ΛΎΣΗ

```
n = int(input("Αριθμός: "))
if n < 2:
    print("Δεν είναι πρώτος")
else:
    for d in range(2, n):
        if n % d == 0:
            print(f"Όχι — διαιρείται με το {d}")
            break
    else:
        print("Είναι πρώτος")
```

N	ΑΠΟΤΈΛΕΣΜΑ
1	Δεν είναι πρώτος
17	Είναι πρώτος
91	Όχι — διαιρείται με το 7

Τι χρησιμοποιήσαμε

if/else για τις ειδικές περιπτώσεις · **for** για τους πιθανούς διαιρέτες · **%** για τη διαιρετότητα · **break** + **else** για το «δεν βρέθηκε».

ΒΕΛΤΊΩΣΗ Αρκεί να ψάξεις διαιρέτες μέχρι τη ρίζα του n: **range(2, int(n ** 0.5) + 1)**. Για n = 1.000.003 κάνει 1.000 γύρους αντί για 1.000.000.

Μάντεψε τον αριθμό

ΕΚΦΏΝΗΣΗ

Ο υπολογιστής «σκέφτεται» έναν αριθμό από 1 έως 100. Ο παίκτης μαντεύει και παίρνει απάντηση **μεγαλύτερος / μικρότερος**, μέχρι να τον βρει. Στο τέλος: πόσες προσπάθειες.

ΛΎΣΗ

```
import random

secret = random.randint(1, 100)
tries = 0
guess = 0

while guess != secret:
    guess = int(input("Μάντεψε (1-100): "))
    tries += 1
    if guess < secret:
        print("Μεγαλύτερος!")
    elif guess > secret:
        print("Μικρότερος!")

print(f"Μπράβο! Το βρήκες σε {tries} προσπάθειες.")
```

random.randint(1, 100)

Τυχαίος ακέραιος από 1 έως 100, **μαζί** με τα δύο άκρα (σε αντίθεση με το range).

Η έξυπνη στρατηγική

Μάντευε πάντα **τη μέση** του διαστήματος που απομένει (50, μετά 25 ή 75…). Βρίσκεις κάθε αριθμό σε **το πολύ 7** προσπάθειες.

ΠΡΏΤΟ ΠΑΙΧΝΊΔΙ Ο βρόχος «περίμενε κίνηση → έλεγξε → απάντησε» είναι η ίδια ιδέα με τον βρόχο ενός παιχνιδιού στο pygame.

Μενού κονσόλας

```
while True:
    print("\n1. Προπαίδεια")
    print("2. Έλεγχος πρώτου")
    print("0. Έξοδος")
    choice = input("Επιλογή: ")

    if choice == "1":
        n = int(input("Αριθμός: "))
        for i in range(1, 11):
            print(f"{n} x {i} = {n * i}")
    elif choice == "2":
        n = int(input("Αριθμός: "))
        prime = n >= 2
        for d in range(2, n):
            if n % d == 0:
                prime = False
                break
        print("Πρώτος" if prime else "Όχι πρώτος")
    elif choice == "0":
        print("Αντίο!")
        break
    else:
        print("Λάθος επιλογή")
```

ΕΚΦΏΝΗΣΗ

Ένα πρόγραμμα που δείχνει μενού, εκτελεί την επιλογή και **ξαναδείχνει το μενού**, μέχρι ο χρήστης να δώσει 0.

Τα πάντα μαζί

while True + **break** για το μενού, **if/elif/else** για την επιλογή, **for** μέσα στις επιλογές.

Γιατί "1" με εισαγωγικά

Η επιλογή μένει **κείμενο**. Έτσι, αν ο χρήστης γράψει «α», πάει στο else αντί να «σκάσει» το int().

ΕΠΈΚΤΑΣΗ Πρόσθεσε επιλογή **3. Μάντεψε τον αριθμό**: αντέγραψε τον κώδικα του Εργαστηρίου 3 μέσα σε ένα νέο elif.

Τι κρατάμε

if / elif / else

Ελέγχει **από πάνω προς τα κάτω** και εκτελεί **μόνο τον πρώτο** κλάδο που ισχύει. Η σειρά μετράει.

while

Όσο ισχύει η συνθήκη. Για άγνωστο πλήθος. Τρία μέρη: αρχικοποίηση, συνθήκη, μεταβολή.

for

Για κάθε στοιχείο: range ή κείμενο. Το range **δεν περιλαμβάνει** το τέλος.

Με **break** βγαίνεις, με **continue** πηδάς έναν γύρο, και το **else** του βρόχου σημαίνει «τελείωσα χωρίς break». Σήμερα έγραψες τέσσερα προγράμματα που **αποφασίζουν** και **επαναλαμβάνουν**.

Επόμενη φορά · Δευ 19/10

Λίστες και πλειάδες: πώς κρατάμε πολλές τιμές σε μία μεταβλητή — και πώς τις διατρέχουμε με το for που μάθαμε σήμερα.

ΓΙΑ ΤΟ ΣΠΊΤΙ Διάβασε θετικό ακέραιο και εμφάνισε **πόσα ψηφία** έχει και **το άθροισμά τους** (π.χ. 4721 → 4 ψηφία, άθροισμα 14). Με while, // και %.